CÓDIGOCLARO · MATERIAL DE ESTUDIO

Java y programación orientada a objetos

Cuaderno DAM/DAW · Java SE 21 · API estándar, sin funciones preview.

Del código fuente a la JVM

Objetivo: Explicar qué comprueba javac y qué ejecuta la JVM al arrancar un proyecto.

Java comprueba sintaxis y tipos al compilar; normalmente produce bytecode que la JVM carga y ejecuta. Separar compilación y ejecución ayuda a localizar un error: un símbolo desconocido se corrige antes de ejecutar, mientras que una clase no encontrada apunta a ruta o classpath. El ejemplo evita características preview y funciona con Java SE 21.

Código · Java

```
public class Main {
  public static void main(String[] args) {
    System.out.println("DAM listo");
  }
}
// javac Main.java
// java Main
```

Práctica: Compila Main.java y ejecútalo desde otra carpeta. ¿Qué debe estar disponible para que Java encuentre Main?

Respuesta razonada: javac crea Main.class en el directorio actual; java debe recibir un classpath que incluya ese directorio. Un error de compilación no se arregla cambiando la ejecución y viceversa.

Tipos y decisiones

Objetivo: Elegir un tipo adecuado y expresar explícitamente las condiciones de una matrícula.

Java es estáticamente tipado: una condición debe producir boolean. Los números no se convierten implícitamente en verdadero o falso. Usa enteros para contadores; cuando el enunciado requiera importes exactos, evita suponer que double representa cada decimal con exactitud. Prueba tanto el límite como un valor justo a cada lado.

Código · Java

```
int plazas = 0;
boolean admiteAlta = plazas > 0;
String estado = admiteAlta ? "abierta" : "completa";
System.out.println(estado);
```

Práctica: Prueba plazas 0, 1 y -1. ¿Qué valor incumple el dominio aunque el condicional lo clasifique?

Respuesta razonada: Cero y -1 hacen que admiteAlta sea falso; solo -1 es un estado de dominio inválido si las plazas nunca pueden ser negativas. Valida esa regla al recibir el dato, no la escondas en la condición.

Métodos y sobrecarga

Objetivo: Diseñar una operación con contrato verificable y distinguir sobrecarga de retorno.

La firma de un método depende del nombre y los parámetros, no solo del tipo de retorno. Un método debe expresar una responsabilidad, validar entradas y devolver un resultado comprensible. Los argumentos se pasan por valor; en un parámetro de referencia se copia la referencia, por lo que el objeto compartido aún puede observar cambios.

Código · Java

```
static double media(int total, int recuento) {
  if (recuento <= 0) throw new IllegalArgumentException();
  return (double) total / recuento;
}
```

Práctica: Añade una sobrecarga que acepte un array y define su comportamiento si está vacío.

Respuesta razonada: La sobrecarga acepta int[] y suma sus elementos. Para el array vacío, rechaza la entrada o devuelve un resultado explícito; dividir entre cero no es una política válida implícita.

Arrays y límites

Objetivo: Recorrer notas sin acceder fuera de los límites ni perder el caso de array vacío.

Los arrays tienen longitud fija y los índices comienzan en cero. Para n elementos, el último índice es n-1 y el bucle habitual usa i < length. El for-each reduce errores si no necesitas la posición. Comprueba la longitud antes de calcular una media: un array vacío no tiene denominador válido.

Código · Java

```
int[] notas = {7, 8, 6};
if (notas.length == 0) {
  throw new IllegalArgumentException("No hay notas");
}
int suma = 0;
for (int nota : notas) suma += nota;
double media = (double) suma / notas.length;
```

Práctica: ¿Qué ocurre si sustituyes i < notas.length por i <= notas.length?

Respuesta razonada: El bucle intenta leer el índice length, fuera del array, y lanza ArrayIndexOutOfBoundsException. La media necesita además tratar length igual a cero.

Clases e invariantes

Objetivo: Evitar que una cuenta de prácticas tenga un saldo inicial inválido.

Una clase agrupa estado y operaciones. Los campos privados impiden modificaciones arbitrarias desde fuera; el constructor y los métodos deben preservar las invariantes. Si el saldo no puede ser negativo, valida al construir y antes de retirar. Si una operación se rechaza, el estado debe conservarse sin cambios.

Código · Java

```
final class Cuenta {
  private int saldo;
  Cuenta(int inicial) {
    if (inicial < 0) throw new IllegalArgumentException();
    saldo = inicial;
  }
  int saldo() { return saldo; }
}
```

Práctica: Define qué debe ocurrir al retirar más dinero del saldo.

Respuesta razonada: La operación debe rechazar el retiro antes de modificar saldo. Así el objeto no atraviesa un estado inválido y el llamador distingue una operación imposible de una retirada correcta.

Constructores y this

Objetivo: Usar constructores para crear productos válidos y delegar la inicialización sin duplicarla.

Un constructor puede delegar en otro de su clase mediante this(...), que debe ser la primera instrucción. Los campos final se asignan durante la inicialización. Validar nombre y cantidad al crear el objeto evita que otros métodos reciban instancias parcialmente válidas.

Código · Java

```
final class Producto {
  final String nombre;
  final int unidades;
  Producto(String nombre) { this(nombre, 0); }
  Producto(String nombre, int unidades) {
    if (nombre.isBlank() || unidades < 0) throw new IllegalArgumentException();
    this.nombre = nombre; this.unidades = unidades;
  }
}
```

Práctica: ¿Debe crearse Producto con nombre vacío o con unidades negativas?

Respuesta razonada: No. Ambos argumentos infringen el contrato, por lo que se rechazan antes de asignar los campos. this.nombre identifica el campo frente al parámetro del mismo nombre.

Herencia y composición

Objetivo: Decidir si una regla variable debe heredar de una clase o delegarse a un colaborador.

Herencia expresa una relación de sustitución: el subtipo debe mantener las garantías de la base. Composición delega comportamiento y suele permitir intercambiar políticas sin ampliar la jerarquía. No heredes solo para reutilizar unas líneas. El diseño más claro es el que conserva el contrato y limita dependencias.

Código · Java

```
interface Descuento { int aplicar(int precio); }
final class PrecioFinal {
  private final Descuento descuento;
  PrecioFinal(Descuento descuento) { this.descuento = descuento; }
  int calcular(int base) { return descuento.aplicar(base); }
}
```

Práctica: ¿Qué ventaja tiene inyectar una política de descuento frente a crear una subclase por campaña?

Respuesta razonada: La política se puede sustituir en pruebas o en ejecución sin multiplicar subclases. Herencia encaja cuando el subtipo puede reemplazar a la base sin cambiar la expectativa del llamador.

Interfaces y despacho dinámico

Objetivo: Procesar avisos de distintos tipos mediante un contrato común.

Una interfaz declara capacidades compartidas sin exigir que las clases hereden de una misma base. Al llamar a un método mediante la interfaz, Java selecciona en ejecución la implementación del objeto real. Una clase abstracta puede compartir estado o código, pero no hace falta si solo se comparte un contrato.

Código · Java

```
interface Aviso { String texto(); }
record Nota(String texto) implements Aviso {}
record ErrorCarga(String texto) implements Aviso {}
class PresentadorAvisos {
  static void mostrar(Aviso aviso) {
    System.out.println(aviso.texto());
  }
}
```

Práctica: Añade AvisoVacio y pásalo a mostrar. ¿Qué consumidor debe cambiar?

Respuesta razonada: Si AvisoVacio implementa la interfaz, mostrar no cambia. La implementación nueva debe cumplir texto(); el consumidor programa contra el contrato y no necesita ramificaciones por clase.

Colecciones y garantías de orden

Objetivo: Elegir entre List, Set y Map según orden, duplicados y búsquedas por clave.

List conserva posiciones y repetidos; Set representa pertenencia sin duplicados; Map asocia claves únicas a valores. HashMap no promete orden de iteración. Usa LinkedHashMap para orden de inserción y TreeMap para orden por clave. Escribe pruebas contra la garantía requerida y no contra un orden incidental.

Código · Java

```
Map<String, Integer> stock = new LinkedHashMap<>();
stock.merge("cable", 1, Integer::sum);
stock.merge("cable", 1, Integer::sum);
System.out.println(stock.get("cable")); // 2
```

Práctica: Cuenta artículos repetidos y presenta las claves alfabéticamente. ¿Qué estructuras usarías?

Respuesta razonada: Un Map acumula frecuencias; TreeMap ordena por clave al presentar. HashMap puede ser suficiente para contar, pero su orden no debe formar parte del resultado esperado.

Excepciones y ficheros

Objetivo: Leer un fichero UTF-8 y comunicar un error sin fingir una lectura correcta.

Path representa una ruta y Files ofrece operaciones de lectura. Las APIs de alto nivel gestionan el cierre; try-with-resources cierra recursos abiertos aunque ocurra una excepción. Fija la codificación del texto y captura IOException solo donde puedas aportar contexto o una recuperación útil.

Código · Java

```
Path ruta = Path.of("notas.txt");
try {
  for (String linea : Files.readAllLines(ruta, StandardCharsets.UTF_8))
    System.out.println(linea);
} catch (IOException ex) {
  System.err.println("No se pudo leer: " + ex.getMessage());
}
```

Práctica: El fichero no existe. ¿Qué información debe recibir el usuario y qué detalle registrarías?

Respuesta razonada: Informa que la lectura ha fallado y registra la causa para depuración; no devuelvas contenido inventado. UTF-8 evita depender de la codificación por defecto del sistema.

Fuentes oficiales

Java Language Specification, Java SE 21 · Dev.java: programación orientada a objetos · API Java SE 21: colecciones · API Java SE 21: Files · JUnit 5.14.4 · User Guide